

System Implementation

FloodGuard Task 2: Serverless and Microservices Layer

Samir Pokhrel (NP0698[##]) · Group G[GROUP NUMBER] · CT071-3-3-DDAC

System Implementation

The Task 2 serverless layer was provisioned entirely through the AWS Command Line Interface so that each step is reproducible and auditable. Table 1 lists what was built. The Task 1 server tier, that is the virtual private cloud, the relational database, both Elastic Beanstalk environments and the content delivery distribution, was retained unchanged and enhanced rather than replaced.

Table 1
Resources Provisioned in the Task 2 Serverless Layer

Service	Resource	Purpose
DynamoDB	fg-weather-snapshots	Append-only weather time series, 30-day expiry
SQS	2 work queues, 2 dead-letter queues	Decoupling and failure isolation
SNS	fg-flood-alerts, fg-ops-alarms	Public fan-out and operator alarms
IAM	6 execution roles, 1 scheduler role	One least-privilege role per function
Lambda	6 functions	The microservices
API Gateway	REST API fg-serverless-api, stage v1	Public entry to two functions
EventBridge	fg-weather-ingest-schedule	Ten-minute ingestion trigger
S3	Object-created notification	Upload is the trigger
CloudFront	/sl/* behaviour	Single front door
CloudWatch	21 alarms, 14-widget dashboard	Monitoring and response

Note. All commands were executed against account 292960609118 in region us-east-1 and verified end to end. The full command sequence is given in the accompanying rebuild guide.

Figure 1
Target Account Confirmation Before Provisioning

SCREENSHOT: aws sts get-caller-identity showing account 292960609118.

Data Layer

DynamoDB Weather Snapshot Table

Partitioned on region and sorted on timestamp, so the latest snapshots for a region resolve in one query. On-demand billing suits periodic rather than sustained ingestion. A time-to-live attribute expires items after thirty days at no write cost, which is what keeps the table inside the free tier.

Figure 2

Creation of the DynamoDB Table and Its Expiry Specification

SCREENSHOT: create-table, wait table-exists, and update-time-to-live together. Must show status moving to ACTIVE, the regionId hash key, the ts range key, and TTL enabled on expiresAt.

SQS Work Queues and Dead-Letter Queues

Four queues rather than two. A dead-letter queue is a separate queue reached by a redrive policy, and that policy is the resilience mechanism. Visibility timeout is six times the longest consumer timeout so SQS cannot redeliver a message that is still being processed, and maxReceiveCount of three routes a repeatedly failing message aside instead of looping forever.

Figure 3

Queue Creation and the Redrive Policy Linking Each Work Queue to Its Dead-Letter Queue

SCREENSHOT: the four queue URLs, then get-queue-attributes showing VisibilityTimeout 360 and the redrive policy naming the dead-letter queue.

SNS Topics

Two topics kept deliberately separate, so an operational alarm storm can never reach residents and a public flood alert is never mistaken for an infrastructure notification.

Figure 4

Creation of the Public Alert and Operations Alarm Topics

SCREENSHOT: both topic ARNs returned by sns create-topic.

Identity and Access

One Execution Role per Function

A single shared role would give fg-report-intake, the only unauthenticated endpoint in the system, permission to publish flood alerts to every subscriber. Six roles confine the blast radius of a bug in any one handler to that handler.

Figure 5

Creation of the Six Lambda Execution Roles

SCREENSHOT: the six role names returned by the iam create-role loop.

Least-Privilege Policy Verification

Each role receives an inline policy naming only the resources that function actually touches.

Figure 6

Evidence of Privilege Separation on the Unauthenticated Intake Function

SCREENSHOT: iam get-role-policy for fg-report-intake-role. The action list must contain secretsmanager:GetSecretValue and nothing else, demonstrating the public endpoint holds no sns:Publish.

Compute Layer

Deployment Packaging

The Node.js 22 runtime already ships the AWS SDK, so five of the six functions are a handler plus a shared helper of a few kilobytes. Only the presign function declares dependencies. Small packages mean short cold starts.

Figure 7

Deployment Archive Sizes Across the Six Functions

SCREENSHOT: the packaging loop output. Five archives at 4 KB and the presign archive at 3.2 MB, showing the dependency cost.

Function Creation

Timeouts are set per workload. Ingestion makes ten sequential outbound calls and receives 300 seconds; an endpoint-fronted function receives 30. Active tracing is enabled on all six, which is what makes the later service map possible.

Figure 8

Creation of the Six Lambda Microservices With Active Tracing

SCREENSHOT: the six JSON responses, each showing nodejs22.x, arm64, its timeout, and Tracing: Active.

Event Source Mappings and Concurrency Allocation

This account permits ten concurrent Lambda executions, not the default thousand. An uncapped queue consumer scales until it owns all of them and starves the public request path, which API Gateway surfaces to users as a server error. Capping the consumers at three and two reserves five executions for synchronous traffic. Partial batch failure reporting prevents one poison message forcing redelivery of its healthy batch-mates.

Figure 9

Queue Consumers Created With Explicit Concurrency Ceilings

SCREENSHOT: both create-event-source-mapping responses, showing batch sizes 5 and 1 against maximum concurrency 3 and 2.

API Layer

REST API and Resource Tree

REST v1 was selected over HTTP API v2 for one decisive reason: HTTP APIs do not emit X-Ray trace segments, and tracing evidence is required. Resources are rooted at /sl so the distribution can forward with an origin path of /v1 and require no path rewriting.

Figure 10

REST API Identifier and Resource Tree

SCREENSHOT: the generated API identifier and the four resource identifiers. Note the identifier is generated fresh on every rebuild.

Integrations and Invoke Permissions

Lambda proxy integration passes the raw request to the handler, so routing and response shape live in code rather than in gateway mapping templates. The resource-based invoke permission is scoped to this API and this method.

Figure 11

Lambda Proxy Integrations and Scoped Invoke Permissions

SCREENSHOT: the two wiring confirmations, plus the console view at API Gateway, fg-serverless-api, Resources, with the POST method selected and the proxy target visible.

Stage Deployment and Tracing

Figure 12

Stage Deployment With X-Ray Tracing Enabled and a Direct Endpoint Test

SCREENSHOT: the deployment identifier, Tracing: true, and the HTTP 200 from the direct execute-api smoke test. tracingEnabled is the setting HTTP APIs lack entirely.

Event Sources

EventBridge Schedule

This replaces an in-process scheduled task that ran on every instance and would have delivered every flood alert twice after the first autoscaling event. EventBridge fires once regardless of fleet size. Identity and Access Management is eventually consistent and the scheduler validates role assumption up front, so creation is retried; on the verified build it succeeded on the second attempt.

Figure 13

Scheduled Trigger Created and Verified

SCREENSHOT: the attempt counter and the verification showing ENABLED, rate(10 minutes), and the ingest function as target.

S3 Object-Created Notification

The invoke permission must exist before the notification, because attaching it validates that the destination is invocable. Uploading a photograph is itself the trigger; nothing polls and nothing is scheduled.

Figure 14*Bucket Notification Wiring Uploads to the Verification Function*

SCREENSHOT: the notification verification showing s3:ObjectCreated:* filtered to the reports/ prefix.

Front Door***CloudFront Serverless Behaviour***

Adding the /sl/* behaviour completes a single hostname with three origins, so every browser request is same-origin and no preflight occurs anywhere in the system.

Figure 15*Distribution Updated With the API Gateway Origin*

SCREENSHOT: the update response showing InProgress, three origins and three cache behaviours. Propagation takes five to ten minutes.

Monitoring***Alarms***

Twenty-one alarms, every one publishing to the operations topic. Metrics without alarm actions is observation without response. The dead-letter queue alarms matter most: a message there means work was permanently dropped.

Figure 16*Alarm Inventory in a Non-Alarming State*

SCREENSHOT: the table of 21 alarms, all reporting OK.

Dashboard

Fourteen widgets laid out along the request path, ending in a log query of recent pipeline errors across all six functions.

Figure 17*Monitoring Dashboard Populated Under Load*

SCREENSHOT: the FloodGuard-Task2 dashboard, time range one hour, captured after the verification run so widgets contain data. Take the upper and lower halves separately so axis labels stay readable.

Verification

An automated check exercises every hop of both pipelines and issues 240 requests so latency percentiles are meaningful rather than idle.

Figure 18

End-to-End Verification of the Rebuilt Serverless Layer

SCREENSHOT: the verification summary. The verified rebuild returned 18 passed, 0 failed, covering routing, the 401 on the internal interface, input rejection at 400 and 415, upload verification, scheduled ingestion, empty dead-letter queues, duplicate suppression and recorded traces.

Figure 19

X-Ray Service Map Spanning Both Tiers

SCREENSHOT: the service map at a one-hour range, showing the API Gateway stage node, the Lambda nodes and the application server node connected. The presence of the gateway node is the payoff for choosing REST over HTTP API.

Figure 20

Object Tags Written by the Image Verification Function

SCREENSHOT: get-object-tagging on the most recent upload, showing verification set to verified and the detected media type. Proof the object notification fired and the file signature check ran.

Microservice Source and Interactions

The brief requires code evidence of how each service is structured and how the services communicate. The functions never share a database: the monolith owns the relational store, the serverless tier owns DynamoDB and S3, and the two talk over queues and an authenticated internal interface.

Figure 21

Source Layout of the Six Microservices

SCREENSHOT: `find infra/lambda -type f | sort`. Six independent function directories plus `_shared`, each with its own `index.mjs`. One directory per service is what makes them independently deployable.

Figure 22

Weather Ingestion Fanning Work Out to Amazon SQS

SCREENSHOT: `infra/lambda/fg-weather-ingest/index.mjs`, lines 93 to 103, in a code editor. Shows `SendMessageBatchCommand` chunking one message per region in groups of ten, the batch API limit. This function performs input and output only; scoring is deliberately somebody else's job.

Figure 23*Flood Forecast Consuming the Queue With Partial Batch Failure Reporting*

SCREENSHOT: `infra/lambda/fg-flood-forecast/index.mjs`, lines 115 to 131. Shows failed message identifiers collected into `batchItemFailures` and returned. Without this, one poison message forces redelivery of its nine healthy batch-mates.

Figure 24*Alert Dispatch: Idempotent Creation Then Fan-Out to Amazon SNS*

SCREENSHOT: `infra/lambda/fg-alert-dispatch/index.mjs`, lines 35 to 64. Shows the duplicate check returning early before `PublishCommand` runs. SQS delivers at least once, so a redelivered message must not notify every resident twice.

Figure 25*Image Verification Reading the File Signature and Tagging the S3 Object*

SCREENSHOT: `infra/lambda/fg-image-process/index.mjs`, lines 20 to 60. Shows the magic-number table, the sniff function, and `PutObjectTaggingCommand`. A presigned URL pins a declared content type but S3 never checks the bytes, so this is the server-side check.

Figure 26*Shared Helper Authenticating Service-to-Service Calls to the Monolith*

SCREENSHOT: `infra/lambda/_shared/common.mjs`, lines 39 to 70. Shows `internalFetch` attaching the shared key header. This is the boundary that lets the functions write to the relational store without ever opening a database connection. Ensure no resolved key value is visible in the captured output.